Internship Daily Documentation

Day 7 — Monday, May 4, 2026

Smart Internship & Hiring Portal | Intern: Mageswaran G

Field	Details
Date	Monday, May 4, 2026
Intern	Mageswaran G
Project	Smart Internship & Hiring Portal
Mentor	Sandiya
Module	Module 2 — User Profile Management (Day 1 of 4)
Day Focus	Backend: Profile APIs + Validation + Security
Final Status	All profile APIs complete, tested, and pushed to GitHub

1. SOD (Start of Day) — Morning Plan

Day 7 marks the start of Module 2: User Profile Management. Before starting, the week plan was finalized based on combined advice from ChatGPT and Claude. ChatGPT gave an important warning about Module 1 UI over-engineering and emphasized functionality first.

Week Plan Agreed (Monday to Thursday):

Day	Focus	Goal
Monday (Day 7)	Backend Profile APIs	GET + PUT profile, validation, security
Tuesday (Day 8)	Resume Upload	Multer setup, file upload API
Wednesday (Day 9)	Frontend Profile Pages	Candidate + Company profile UI
Thursday (Day 10)	Testing + Docs	End to end testing, documentation

SOD Mail Sent to Sandiya:

Subject: SOD Update – Day 7 | Module 2: User Profile Management

Planned tasks: Extend User model, build GET and PUT profile APIs, add Zod validation, test in Thunder Client. Priority: complete backend before moving to frontend.

2. User Model Extension

The User model was extended with 12 new profile fields. Instead of creating a separate profile collection, all fields were added to the existing User model. This is simpler and faster for Module 2.

File: backend/models/User.js

Field	Type	Default	Purpose	Used By
bio	String	''	Short about me text	Both
location	String	''	City or state	Both
phone	String	''	Contact number	Both
profilePhoto	String	''	File path for photo	Both
skills	[String]	[]	Array of skill names	Candidate only
education	String	''	Education background	Candidate only
experience	String	''	Work experience	Candidate only
resumeUrl	String	''	Uploaded resume path	Candidate only
companyName	String	''	Company name	Company only
companyWebsite	String	''	Company website URL	Company only
industry	String	''	Industry type	Company only

All fields use default: '' or default: [] so existing users are not affected. No crash on old data.

3. Validation Schema — validators.js

File: backend/utils/validators.js — updateProfileSchema added below existing signup and login schemas.

Why Each Validation Rule Was Chosen:

Field	Validation Rule	Why
bio	.trim().max(500)	Removes spaces. Max 500 chars — short bio only
location	.trim().max(100)	Clean text. City names are short
phone	.regex(/^[0-9+\-()]{7,15}$/)	Only valid phone characters allowed. 7-15 digits
skills	z.array(z.string().trim().max(50)).max(20)	Each skill trimmed. Max 50 chars per skill. Max 20 skills
education	.trim().max(200)	Clean text. Education is short
experience	.trim().max(200)	Clean text. Brief experience summary
companyName	.trim().max(100)	Company names are short
companyWebsite	.max(200).refine(starts with http)	Must start with http:// or https://. max() BEFORE refine() in Zod v3
industry	.trim().max(100)	Industry type is short

Important: .strict() added to schema — Zod rejects any field not in the schema. This prevents unknown field injection.

4. Service Layer — userService.js

4.1 getProfile Function

Finds user by ID in MongoDB. Returns all fields except password and refreshToken using .select('-password -refreshToken').

4.2 updateProfile Function — Key Design Decisions

Design Choice	What It Does	Why
allowedFields whitelist	Only 9 specific fields allowed	Prevents role, password, refreshToken from being updated
for...of loop	Iterates over update keys	Modern JavaScript. Cleaner than forEach
Empty payload check	Throws 400 if nothing to update	Prevents useless DB writes
new: true	Returns updated document	Mongoose syntax — not returnDocument:'after' (MongoDB driver)
.select('-password -refreshToken')	Hides sensitive fields	Never expose password or tokens in response

5. Controller Layer — userController.js

File: backend/controllers/userController.js — Two new functions added. Old duplicate functions removed.

Function	Route	What It Does
getMyProfile	GET /profile	Calls userService.getProfile with req.user.id from JWT
updateMyProfile	PUT /profile	Calls userService.updateProfile with req.user.id and req.body

Controller is a thin layer — only handles req/res. All business logic stays in service layer.

6. Routes — userRoutes.js

File: backend/routes/userRoutes.js — Clean version with all correct imports.

Method	Route	Middleware Chain	Purpose
GET	/profile	verifyToken → getMyProfile	Get logged in user profile
PUT	/profile	verifyToken → validateMiddleware → updateMyProfile	Update logged in user profile
GET	/	none	Get all users (public for now)

GET	/admin-only	verifyToken → authorizeRole('admin')	Admin only route

7. All Bugs Fixed Today — Complete List

Today had many debugging challenges. Every bug was found, understood, and fixed.

Bug #	Error Message	Root Cause	Fix Applied
1	require is not defined in ES module scope	Zod v4 is ESM-only — cannot use require()	npm install zod@3 — downgrade to CommonJS-compatible version
2	userController is not defined	userRoutes.js imported individual functions, not full controller object	Changed to: const userController = require('../controllers/userController')
3	validateMiddleware is not a function	validateMiddleware.js was exporting as object {validateMiddleware:fn}	Changed to: module.exports = validateMiddleware (direct function export)
4	validate is not a function	authRoutes.js imported { validate } but file exports validateMiddleware	Fixed import name to match export name
5	xssClean is not defined	server.js imported { xssClean } but xssMiddleware.js exports xssMiddleware	Fixed import and usage to use xssMiddleware
6	filteredData already declared	updateProfile function had const filteredData declared twice	Replaced entire updateProfile with clean single declaration
7	refine().max() is not a function	In Zod v3, .max() must come BEFORE .refine()	Swapped order: .max(200).refine(...) — max first, then refine
8	Expected array, received object	Thunder Client body was set to XML tab not JSON tab	Changed Thunder Client body type to JSON
9	Connection refused / 0B response	server.js crashed but Thunder Client tried to send	Fixed server crash first, then retested
10	returnDocument deprecated warning	Used MongoDB driver syntax in Mongoose	Changed to { new: true } which is correct Mongoose syntax

8. Security — 3-Layer Protection System

Today's backend has 3 separate layers of protection working together:

Layer	Where	What It Checks	Example
Layer 1 — Zod Validation	validateMiddleware	Field types, formats, max lengths, .strict() blocks unknown keys	role:admin → rejected immediately
Layer 2 — allowedFields Whitelist	userService.updateProfile	Even if Zod passes, only 9 fields allowed through	Any unlisted field silently ignored
Layer 3 — XSS Sanitization	xssMiddleware	All string input cleaned — script tags converted to safe text	<script> → <script>

ChatGPT security rating after today: 9.2/10. 'Above internship level. You are no longer in learning CRUD — you are building a secure API layer.'

9. Thunder Client Test Results

Test #	Method	Input	Expected	Result
1	GET /profile	Bearer token	200 — full profile with all 12 fields	PASS ✅
2	PUT /profile	{bio, location, skills}	200 — profile updated	PASS ✅
3	PUT /profile	{role: 'admin'}	400 — Unrecognized key(s): 'role'	PASS ✅
4	PUT /profile	{}	400 — No valid fields to update	PASS ✅
5	PUT /profile	{bio: '<script>alert(1)</script>'}	200 — stored as <script> (sanitized)	PASS ✅
6	PUT /profile	{phone: 'abc'}	400 — Enter a valid phone number	PASS ✅
7	PUT /profile	{companyWebsite: 'mysite.com'}	400 — Website must start with http://	PASS ✅

All 7 tests passing. Security layer proven working end to end.

10. Key Concepts Learned Today

Concept	Simple Explanation
CommonJS vs ESM	require() = CommonJS. import = ESM. Cannot mix them. Zod v4 is ESM-only so must use Zod v3 for CommonJS projects.

Zod .strict()	Rejects any field not defined in schema. Very powerful for security. Role injection impossible.
Zod chain order	In Zod v3, .max() must come BEFORE .refine(). After refine, no more string methods.
allowedFields whitelist	Never trust req.body directly. Always filter — only pass known safe fields to database.
for...of vs forEach	for...of is modern JS — cleaner and slightly faster. Use it for simple loops.
new: true in Mongoose	Correct Mongoose syntax to get updated document back. returnDocument is MongoDB driver syntax.
Promise.all	Runs multiple async operations at same time. Both must finish before continuing.
Content-Type header	Without this header, Express does not know body is JSON. Always add when testing PUT/POST.
XSS sanitization	Convert dangerous HTML characters to safe text. <script> becomes <script>. Shown in DB.
Empty payload check	Always check if anything to update before hitting DB. Saves resources. Returns clear error.
Module export styles	module.exports = fn (direct). module.exports = { fn } (object). Import must match export style.

11. Files Changed Today

File	What Changed
backend/models/User.js	Added 12 new profile fields with defaults
backend/utils/validators.js	Added updateProfileSchema with all validations. Fixed Zod v3 chain order.
backend/services/userService.js	Added getProfile. Added updateProfile with allowedFields whitelist. Removed duplicate getUserById.
backend/controllers/userController.js	Added getMyProfile and updateMyProfile. Removed duplicate getProfile.
backend/routes/userRoutes.js	Complete rewrite — clean imports, no duplicates, correct middleware chain
backend/routes/authRoutes.js	Fixed validateMiddleware import name (was validate, now validateMiddleware)
backend/middleware/validateMiddleware.js	Changed to call next(err) instead of returning response directly
backend/middleware/xssMiddleware.js	Added Array.isArray check before object check — fixes array sanitization
backend/server.js	Fixed xssMiddleware import name (was xssClean). Updated app.use() call.
backend/package.json	Downgraded zod from v4.3.6 to v3.x — required for CommonJS compatibility

12. GitHub Commits Today

Commit Message	Files Changed	What Was Done
Module 2: Profile GET and PUT APIs complete and tested	5 files	First working version of profile APIs
Module 2: Profile API improvements - validation, security hardening	validators.js, userService.js	Added skills limits, phone regex, website validation, empty check
Fix: Mongoose findByIdAndUpdate use new:true not returnDocument	userService.js	Fixed Mongoose syntax for returned document
Module 2: Unique validations - phone regex, URL check, skills trim	validators.js, userService.js	Final validation improvements

13. EOD Summary

Task	Status
Week plan finalized (Mon-Thu)	Done ✅
SOD mail sent to Sandiya	Done ✅
User model extended with 12 profile fields	Done ✅
Zod updateProfileSchema created	Done ✅
GET /profile API built and tested	Done ✅
PUT /profile API built and tested	Done ✅
Security — role injection blocked	Done ✅
Security — empty payload blocked	Done ✅
Security — XSS sanitization verified	Done ✅
Security — invalid phone blocked	Done ✅
Security — invalid URL blocked	Done ✅
All 7 Thunder Client tests passing	Done ✅
10 bugs found and fixed	Done ✅
GitHub pushed with clean commits	Done ✅
EOD mail sent to Sandiya	Done ✅

14. Tomorrow — Day 8 Plan (Tuesday)

- Install Multer — npm install multer
- Create backend/uploads/resumes/ folder
- Build POST /api/v1/users/upload-resume API
- PDF only — file type validation

- Max 5MB file size limit
- Store file path in resumeUrl field in User model
- Test resume upload in Thunder Client
- Start frontend profile page (if time permits)